



Gruppo SkyNet
skynetunigroup@gmail.com

Progettazione Frontend (MVP)

Versione	<i>1.0</i>
Uso	<i>Interno</i>
Redattori	<i>Dario Pirrone</i>
Verifica	<i>Nome Cognome</i>
Approvazione	<i>Nome Cognome</i>

Indice

Indice	2
Elenco delle tabelle	4
Elenco delle figure	4
I Specifica Operativa (UX/UI)	5
1 Perimetro del documento	5
1.1 Obiettivo	5
1.2 Cosa progetto	5
1.3 Cosa non progetto (fuori perimetro MVP)	5
1.4 Mappatura Componenti PoC → Frontend MVP	6
2 Architettura dell'informazione	7
2.1 Ruoli e permessi	7
2.2 Mappa delle pagine e delle rotte	7
2.3 Credenziali e servizi esterni	9
3 Flussi utente per ruolo	9
3.1 Notazione	9
3.2 Flusso trasversale: accesso e configurazione iniziale	9
3.3 Selezione ed avvio multiplo delle operazioni (trasversale)	10
3.4 Flusso Sviluppatore — Agente Docs	10
3.5 Flusso Security Auditor / Team Lead — Agente Security	11
3.6 Flusso Project Manager — Agente Changelog	11
3.6.1 Dipendenza di esecuzione tra Changelog tecnico e di business	11
3.7 Flusso trasversale: consultazione dei report pregressi	13
3.8 Flusso trasversale: rilevamento di una credenziale non più valida	13
4 Wireframe descrittivi delle schermate	14
4.1 Notazione	14
4.2 Registrazione — /register	14
4.3 Login — /login	14
4.4 Servizi connessi — /credentials	15
4.5 Selezione del contesto — /select	15
4.6 Avvio operazioni — /run	16
4.7 Dashboard — /tasks	16
4.8 Dettaglio report — /reports/:id	17
4.9 Storico report — /reports	17
4.10 Intestazione applicativa	18
5 Design System e componenti condivisi	18
5.1 Stack di implementazione	18

5.2	Token visivi	18
5.3	Catalogo dei componenti condivisi	19
5.4	Stati interattivi comuni	20
5.5	Accessibilità (A11y)	20
6	Stato applicativo	20
6.1	Principio generale	20
6.2	<code>sessionStore</code> — Identità e credenziali	21
6.3	<code>selectionStore</code> — Contesto operativo	22
6.4	<code>tasksStore</code> — Stato vivo delle operazioni	22
6.5	Dati intenzionalmente non globali	23
6.6	Guardie di rotta	23
7	Contratto API consumato	24
7.1	Principi generali del contratto	24
7.2	Endpoint REST pubblici	24
7.3	Canale realtime (WebSocket)	26
7.4	Schemi dei DTO principali	27
7.5	API interne	28
8	Gestione degli errori	28
8.1	Due famiglie di errore	28
8.2	Pattern per gli errori di validazione e configurazione	28
8.3	Pattern per gli errori di esecuzione di un'operazione	29
8.3.1	Errore di orchestrazione o routing	30
8.4	Errore di esportazione (Report)	30
9	Tracciamento Requisito → Decisione Frontend	30
9.1	Decisioni Operative e Architetture	30
9.2	Requisiti mappati a livello di Flussi e Wireframe	32
9.3	Requisiti Fuori Perimetro Frontend	32
II	Motivazioni e Decisioni Architetture	32
10	Principi Guida e Vincoli di Progetto	33
11	Decisioni Architetture (ADR-FE) e Motivazioni	34
11.1	Identità, Sessione e Ruoli	34
11.2	Integrazioni e Credenziali	35
11.3	Stack, Design System e Report	35
12	Compromessi e Rischi Accettati	35
13	Vincoli Ereditati dall'Infrastruttura AWS	37
13.1	Origine unica e assenza di policy CORS	37
13.2	Routing lato client e Custom Error Response	37
13.3	Configurazione a build-time per risorse statiche	37

13.4 Requisiti per connessioni WebSocket	38
13.5 Streaming dei file ed Esportazione PDF	38
13.6 Gestione degli errori di rete e Sicurezza Trasversale	38

Elenco delle tabelle

1	Mappatura dei componenti del PoC riutilizzati, estesi o introdotti ex novo per il frontend dell'MVP.	7
2	Mappatura delle operazioni per ruolo operativo.	7
3	Mappa delle rotte del frontend e relative dipendenze API.	9
4	Categorie di token visivi condivisi e ambito di applicazione.	19
5	Endpoint REST pubblici consumati dal frontend, raggruppati per area funzionale.	25
6	Eventi WebSocket emessi dal backend verso il frontend.	26
7	Mappatura degli errori di esecuzione sul componente Stato d'errore . . .	29
8	Mappatura Requisiti - Decisioni operative del Frontend.	32
9	Sintesi dei vincoli di progetto e relativo impatto sulla progettazione Frontend.	34
10	Matrice dei Compromessi e Rischi Accettati per il Frontend MVP.	36
11	Riepilogo dei vincoli tecnici imposti al frontend dalle decisioni infrastrutturali.	39

Elenco delle figure

1	Mappa di navigazione principale e reindirizzamenti condizionali del frontend.	8
2	Diagramma di sequenza delle interazioni intermedie per l'Agente Changelog.	12
3	Architettura degli store applicativi, comunicazione di rete ed effetti collaterali.	21
4	Ciclo di vita di una Task. La richiesta di input (pendingInput) non costituisce un nuovo stato, ma una condizione sospensiva interna allo stato RUNNING	22

Parte I

Specifica Operativa (UX/UI)

Questa parte contiene esclusivamente le specifiche implementative, la mappa delle pagine, i flussi operativi e le regole di visibilità del frontend. I principi guida, le motivazioni delle scelte e le decisioni architetturali (ADR-FE) sono documentati nella Parte II.

1 Perimetro del documento

1.1 Obiettivo

Il presente documento definisce la progettazione UX/UI del frontend di **Code Guardian** per la release MVP, a partire dai requisiti descritti nell'Analisi dei Requisiti (AdR). L'architettura e lo stack tecnologico validati nel Proof of Concept (PoC) vengono riutilizzati come base implementativa; le funzionalità non coperte in quella fase (autenticazione, storico, esportazione PDF) vengono qui progettate ex novo. Tutte le scelte operative rispettano i vincoli definiti nel documento di Progettazione Infrastruttura AWS.

1.2 Cosa progetto

- L'intero flusso di autenticazione e la gestione del ruolo operativo dell'utente (**Sviluppatore, Security Auditor, Project Manager**), definito in fase di registrazione.
- La configurazione delle credenziali dei servizi esterni (GitHub e Task Management unificati).
- La selezione del contesto operativo (repository, riferimento di base, ambito di analisi).
- La configurazione, l'avvio e il monitoraggio delle operazioni degli agenti tramite canale realtime.
- La visualizzazione del report strutturato, estesa con il collegamento alla *Pull Request* generata dal backend e l'esportazione in formato PDF.
- Lo storico dei report pregressi.
- Le regole di visibilità e abilitazione delle interfacce in base al ruolo dell'utente autenticato.

1.3 Cosa non progetto (fuori perimetro MVP)

I seguenti requisiti sono esclusi dal perimetro della presente progettazione:

- **RF.18** — Selezione di una Pull Request come riferimento di base (UC9.2, UC9.5, UC14). Il tipo di riferimento supportato è esclusivamente *Branch* (con hash di Commit opzionale).
- **RF.38, RF.75–RF.78** — Impostazioni di notifica e flussi di invio email (UC21.2, UC30–UC33).
- **RF.79–RF.81** — Gestione e personalizzazione del template del README (UC34, UC34.1–UC34.3). L’Agente Docs opera esclusivamente con il template di default.

1.4 Mappatura Componenti PoC → Frontend MVP

La Tabella 1 mappa le aree funzionali validate nel Proof of Concept rispetto alle azioni implementative previste per il frontend dell’MVP.

Area validata nel PoC	Riuso	Azione per l’MVP
Stack (React, Vite, Tan-Stack Router, pnpm, Zustand)	Totale	Nessuna modifica: stack confermato e utilizzato come base di partenza.
<code>sessionStore</code>	Esteso	Conversione da store di sola presenza del token a store di sessione autenticata (aggiunta utente, JWT, ruolo attivo e stato credenziali).
<code>selectionStore</code>	Totale	Nessuna modifica strutturale: rimozione dell’opzione Pull Request dal tipo di riferimento.
<code>tasksStore</code> + Client WebSocket	Esteso	Integrazione della gestione degli stati interattivi (UC41.1, UC43) per le richieste di input utente. Questo flusso costituisce un’estensione reale rispetto al PoC e richiede l’introduzione di un nuovo evento WS e di un nuovo endpoint REST.
Configurazione Token (GitHub)	Esteso	Evoluzione in schermata generica di “Servizi connessi”, per supportare estensioni future.
Pagina <code>/select</code>	Totale	Riuso dell’albero di selezione contestuale; rimosso il ramo Pull Request.
Pagina <code>/run</code>	Esteso	Aggiunta logica di disabilitazione e filtraggio delle operazioni in base al ruolo attivo.
Pagina <code>/tasks</code> (Dashboard)	Esteso	Supporto alla renderizzazione condizionale per gli stati interattivi (inserimento Sprint ID, conferma metadati).
Rendering del Report	Esteso	Mantenuto il dispatcher a blocchi polimorfi. Il componente <code>Proposal</code> viene esteso promuovendo a elemento primario il link alla Pull Request.
Esportazione report	Esteso	Sostituzione dei formati <code>.md/.json</code> con l’invocazione dell’endpoint backend per il download del file <code>.pdf</code> .

Area validata nel PoC	Riuso	Azione per l'MVP
Autenticazione e Storico	Nuovo	Componenti e viste progettate ex novo, in quanto non necessarie nella validazione tecnologica del PoC.

Tabella 1: Mappatura dei componenti del PoC riutilizzati, estesi o introdotti ex novo per il frontend dell'MVP.

2 Architettura dell'informazione

2.1 Ruoli e permessi

Il sistema prevede tre specializzazioni di Utente Generico: lo **Sviluppatore**, il **Security Auditor / Team Lead** e il **Project Manager**. Il ruolo viene selezionato in fase di registrazione e rimane fisso per l'intera durata dell'account, coerentemente con i requisiti di base del sistema.

Ogni ruolo eredita le funzionalità trasversali (selezione del repository, monitoraggio, visualizzazione ed esportazione dei report) e aggiunge l'accesso privilegiato alle operazioni del proprio dominio, come delineato nella Tabella 2.

Operazione	Attore abilitato	Codice Op.
Generazione/aggiornamento README	Sviluppatore	DOCS_README
Documentazione inline (JSDoc)	Sviluppatore	DOCS_INLINE
Documentazione API	Sviluppatore	DOCS_API
Scansione vulnerabilità OWASP Top 10	Security Auditor / Team Lead	SECURITY_OWASP
Verifica conformità Policy-as-code	Security Auditor / Team Lead	SECURITY_POLICY
Changelog tecnico	Project Manager / Sviluppatore	CHANGELOG_TECHNICAL
Changelog di business	Project Manager	CHANGELOG_BUSINESS

Tabella 2: Mappatura delle operazioni per ruolo operativo.

Regola di visibilità (Filtraggio): La lista delle operazioni disponibili sulla pagina di avvio mostra *esclusivamente* le opzioni abilitate per il ruolo attivo dell'utente. Le operazioni non permesse vengono omesse dall'interfaccia (nessuna visualizzazione in stato disabilitato). Questo filtraggio è applicato nativamente dall'endpoint backend in base al ruolo dichiarato nel token di sessione.

2.2 Mappa delle pagine e delle rotte

L'accesso alle pagine è regolato dallo stato di autenticazione e dallo stato della credenziale GitHub. La Figura 1 illustra la mappa di navigazione principale e i reindirizzamenti

(redirect) condizionali applicati dalle guardie di rotta (dettagliate in §6.6).

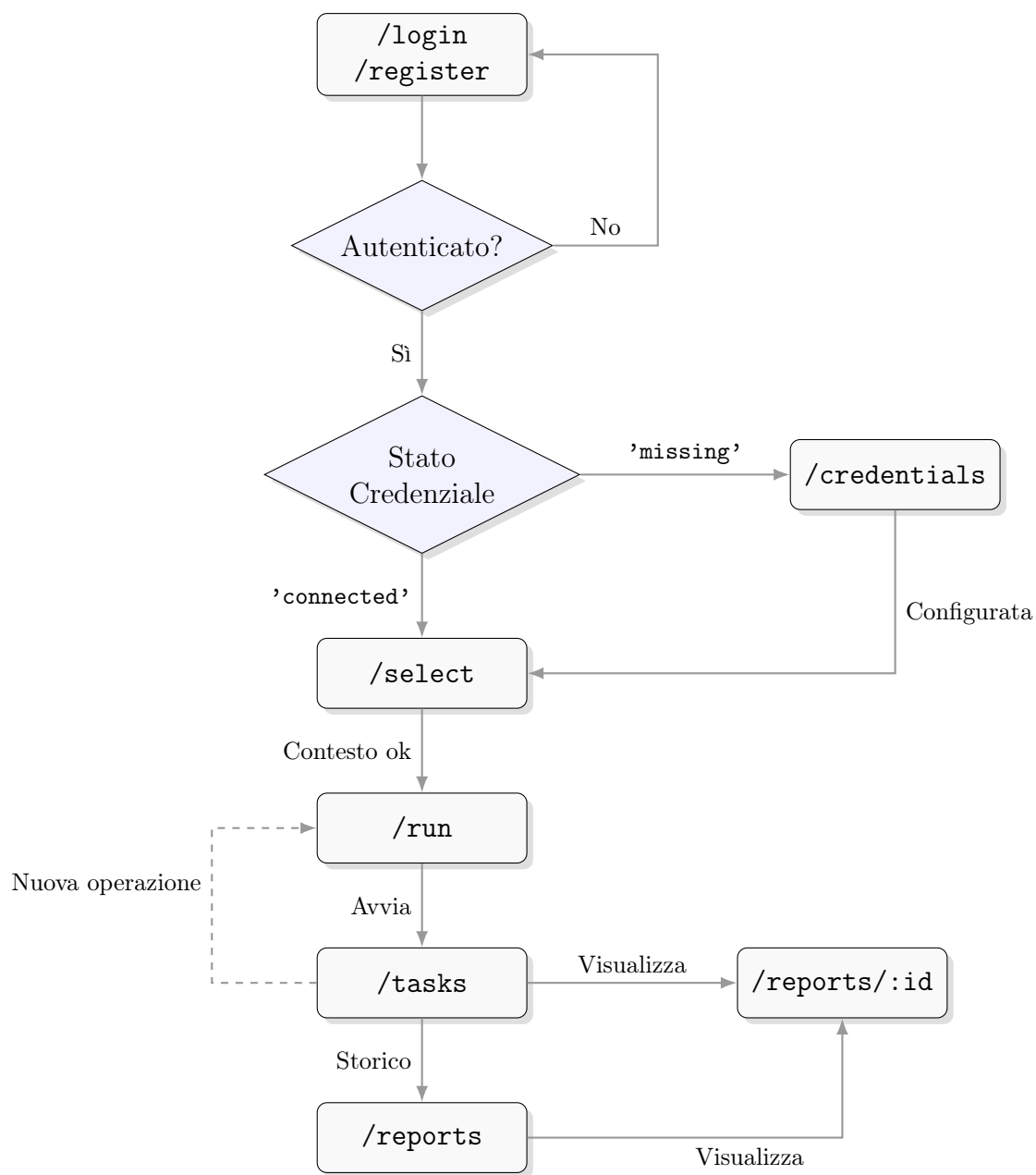


Figura 1: Mappa di navigazione principale e reindirizzamenti condizionali del frontend.

La Tabella 3 definisce le rotte dell'applicazione, i ruoli ammessi e gli endpoint REST consumati da ciascuna di esse.

Pagina	Rotta	Ruoli	Endpoint consumati
Registrazione	/register	Non autenticato	POST /auth/register
Login	/login	Non autenticato	POST /auth/login
Servizi connessi	/credentials	Tutti	POST /credentials, .../validate, GET /credentials

Pagina	Rotta	Ruoli	Endpoint consumati
Selezione contesto	/select	Tutti	GET /repositories, .../refs, .../tree, POST /contexts
Avvio operazioni	/run	Tutti (filtrate)	GET /operations, POST /tasks
Dashboard	/tasks	Tutti	GET /tasks, POST /tasks/:id/cancel, POST /tasks/:id/input, WebSocket
Storico report	/reports	Tutti	GET /reports
Dettaglio report	/reports/:id	Tutti	GET /reports/:id, GET /reports/:id/export?format=pdf

Tabella 3: Mappa delle rotte del frontend e relative dipendenze API.

2.3 Credenziali e servizi esterni

La schermata `/credentials` è implementata come una lista scalabile di `ServiceCredential`. Nell'MVP è presente un unico servizio esterno configurabile: **GitHub**. Questo singolo token assolve a entrambe le necessità descritte nei requisiti (accesso al codice e accesso al sistema di Task Management/Issue).

Invalidazione a runtime: Se durante l'esecuzione di un'operazione il backend rileva che il token GitHub non è più valido (es. scadenza o revoca), l'operazione in corso fallisce con codice `CREDENTIAL_INVALID`. Il frontend intercetta questo stato tramite l'evento `WebSocket`, aggiorna lo stato globale dell'utente a `'invalid'` e mostra un banner persistente che invita a riconfigurare la credenziale su `/credentials`, bloccando nel frattempo l'avvio di nuove analisi.

3 Flussi utente per ruolo

3.1 Notazione

I flussi sono descritti come sequenze ordinate di schermate, per mantenere la tracciabilità rispetto agli scenari principali dell'Analisi dei Requisiti. I punti di diramazione per errore (validazioni fallite, timeout, superamento soglie) non sono ripetuti qui, poiché seguono tutti il pattern di stato interrotto descritto in §8.

3.2 Flusso trasversale: accesso e configurazione iniziale

Comune a tutti e tre i ruoli, precede qualunque operazione degli agenti.

1. L'utente non autenticato arriva su `/login`. Se non possiede un account, passa a `/register`, compila l'anagrafica, le credenziali e seleziona il proprio ruolo operativo iniziale.
2. Al primo login, l'utente privo di credenziali configurate viene reindirizzato a `/credentials`: nessuna operazione degli agenti è raggiungibile senza almeno un servizio connesso e validato.

3. L'utente inserisce il token GitHub; il sistema esegue la validazione sintattica locale e la chiamata di test prima di persistere la credenziale.
4. Completata la configurazione, l'utente accede a `/select` per definire il contesto operativo: URL del repository, branch e, opzionalmente, hash del commit e ambito di analisi.
5. Il contesto salvato (`contextId`) è condiviso da tutte le operazioni successive avviate nella stessa sessione di lavoro; l'utente procede a `/run`.

Le credenziali e il contesto restano validi tra una sessione e l'altra: un utente già configurato, ai login successivi, salta direttamente da `/login` a `/run` o a `/select` se desidera cambiare repository.

3.3 Selezione ed avvio multiplo delle operazioni (trasversale)

Questo flusso è indipendente dal ruolo attivo, il quale determina unicamente le operazioni visibili sulla schermata.

1. Su `/run` il sistema recupera ed elenca le operazioni disponibili per il ruolo attivo dell'utente.
2. L'utente seleziona/deseleziona una o più operazioni (selezione multipla supportata).
3. L'utente conferma l'avvio. Il sistema verifica lato client la presenza di almeno un'operazione selezionata e dei prerequisiti di contesto raccolti in §3.2.
4. Il sistema invia `POST /tasks` con l'elenco delle operazioni e reindirizza l'utente a `/tasks`, dove ciascuna operazione avviata compare come elemento indipendente della dashboard, partendo dallo stato di *In attesa* (PENDING).
5. Ogni operazione procede in isolamento, delegando al backend la gestione della concorrenza (le operazioni per lo stesso agente rimarranno *In attesa* finché l'orchestrazione non le sbloccherà). Il fallimento di un'analisi non blocca o invalida le altre.

3.4 Flusso Sviluppatore — Agente Docs

Copre le operazioni di Generazione README, Documentazione inline e Documentazione API. Condividono lo stesso flusso a livello di interfaccia.

1. Lo Sviluppatore segue il flusso comune (§3.2, §3.3), selezionando una o più operazioni del dominio Docs.
2. Su `/tasks`, l'operazione avanza attraverso gli stati *In attesa* → *In elaborazione* → *Completato*, senza ulteriori input richiesti.
3. Al completamento, lo Sviluppatore apre il report. Il corpo del report mostra come elemento primario il collegamento alla Pull Request aperta automaticamente

dal backend; per la documentazione inline, include inoltre gli avvisi di complessità eccessiva come elementi a gravità informativa.

4. Lo Sviluppatore valuta la proposta direttamente su GitHub: l'interfaccia non prevede un passo di approvazione interno.

3.5 Flusso Security Auditor / Team Lead — Agente Security

Copre la Scansione vulnerabilità OWASP e la Verifica conformità Policy-as-code.

1. Il Security Auditor / Team Lead segue il flusso comune, selezionando le operazioni di sicurezza.
2. Per la verifica policy-as-code, nessun input aggiuntivo è richiesto (la presenza del file `POLICY.md` è validata automaticamente dal backend).
3. Al completamento, il report mostra l'elenco dei riscontri come lista strutturata filtrabile per gravità: per ciascun elemento vengono esposti categoria, gravità, riferimento al file/riga e la *remediation* proposta tramite snippet di codice o testo descrittivo.

3.6 Flusso Project Manager — Agente Changelog

È il flusso con la maggiore interattività, in quanto prevede punti di sospensione a metà esecuzione per l'inserimento e la validazione di input da parte dell'utente. La Figura 2 illustra lo scambio di eventi e schermate.

Al termine dell'elaborazione, il Project Manager visualizza il changelog tecnico formattato in Markdown con i collegamenti ipertestuali ai ticket di origine.

3.6.1 Dipendenza di esecuzione tra Changelog tecnico e di business

Coerentemente con la preconditione di UC41 (“L'utente ha selezionato l'operazione 'Generazione changelog tecnico' o 'Generazione changelog di business'”), la selezione di `CHANGELOG_BUSINESS` in `/run` è sufficiente e autonoma: non richiede la co-selezione di `CHANGELOG_TECHNICAL`. L'Orchestratore avvia un'unica Task con `operation: 'CHANGELOG_BUSINESS'` che attraversa due fasi interne allo stesso stato `RUNNING`:

1. Generazione del Changelog tecnico (UC41), il cui esito viene reso disponibile all'utente come report intermedio.
2. Sospensione dell'esecuzione: la Task valorizza `pendingInput` con `kind: 'BUSINESS_CONFIRMATION'` forzando in Dashboard l'espansione dell'area interattiva con i comandi “Accetta e genera Business” / “Annulla” (UC45/UC46) — lo stesso pattern grafico già usato per lo Sprint ID.
3. Alla conferma, la medesima Task riprende l'elaborazione e genera il Changelog di business (UC45), transitando infine a `COMPLETED`.

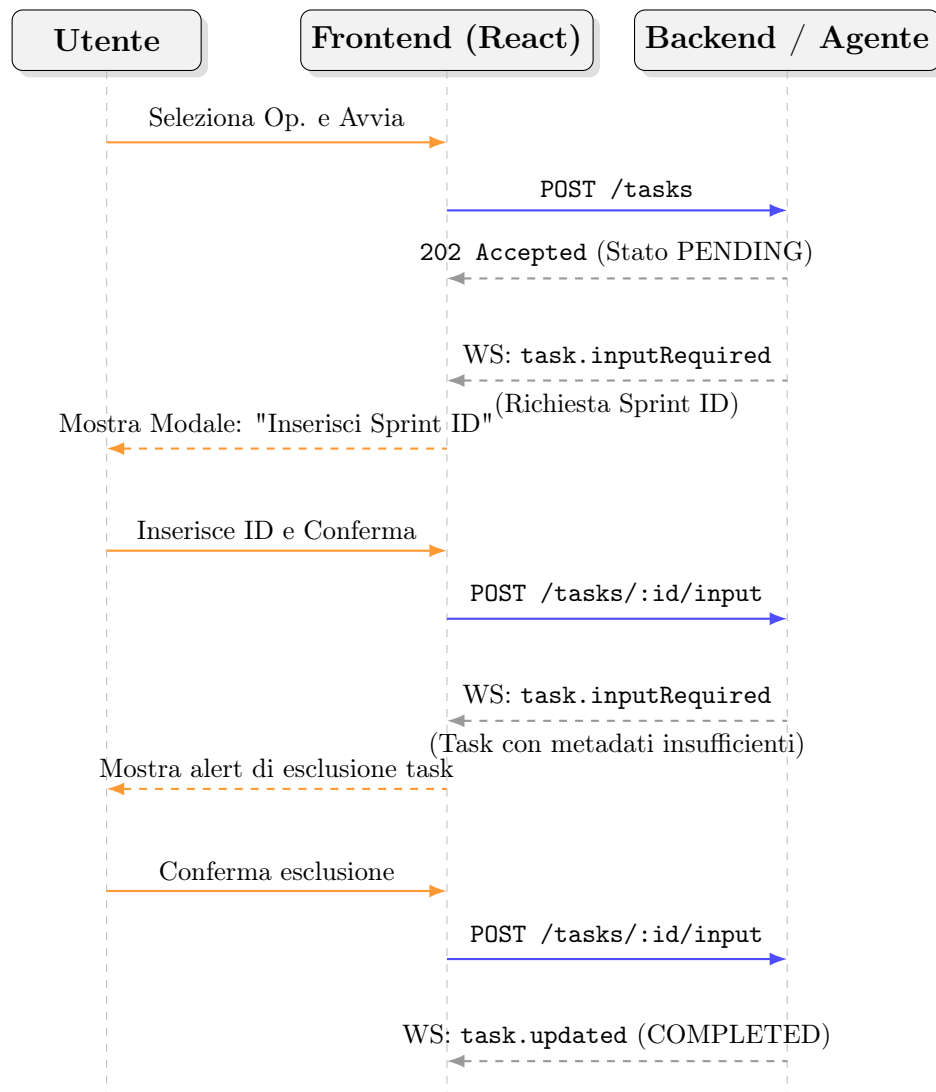


Figura 2: Diagramma di sequenza delle interazioni intermedie per l'Agente Changelog.

In Dashboard, dall'avvio alla conclusione, l'operazione compare quindi come un **singolo elemento**, non come due Task distinte. Se l'utente desidera anche un report tecnico standalone (senza business), seleziona separatamente **CHANGELOG_TECHNICAL**: in tal caso viene creata una seconda Task indipendente, che non ha alcuna relazione con l'eventuale Task **CHANGELOG_BUSINESS** avviata in parallelo (nessuna Task condivide risultati con l'altra; ciascuna esegue la propria generazione tecnica in isolamento).

Nota Implementativa

La scelta di mantenere disaccoppiati i comandi di selezione in UI e delegare l'attesa all'Orchestratore backend evita di inserire rigida logica di business all'interno del client. Il frontend si limita a leggere lo stato `pendingInput` inoltrato tramite WebSocket e a renderizzare le modali di conseguenza, ignorando del tutto le motivazioni sequenziali che lo hanno innescato.

3.7 Flusso trasversale: consultazione dei report pregressi

1. Da qualunque punto dell'applicazione, l'utente autenticato raggiunge `/reports`, che elenca i report salvati con identificativo, titolo, data e ora di generazione.
2. La selezione di un elemento apre `/reports/:id`. La pagina di dettaglio del report è un unico componente condiviso, raggiungibile sia dalla dashboard (a operazione appena completata) sia dallo storico.
3. Da entrambi i punti di ingresso, l'utente può richiedere l'esportazione in PDF, generato lato backend e reso disponibile in download.

3.8 Flusso trasversale: rilevamento di una credenziale non più valida

Questo evento può interrompere qualunque flusso in qualsiasi momento.

1. Durante l'esecuzione di un'operazione, il backend incontra un errore 401/403 nel leggere dal servizio esterno.
2. Il backend marca la Task come **FAILED** con codice **CREDENTIAL_INVALID**; l'evento WebSocket trasporta questa informazione.
3. Il frontend aggiorna la Task interessata come un normale fallimento e imposta lo stato della credenziale globale a `'invalid'` (§2.3).
4. L'interfaccia applicativa mostra un banner persistente su tutte le pagine finché lo stato resta `'invalid'`, offrendo un collegamento diretto a `/credentials`.
5. Ripristinata la connessione, il banner scompare e le guardie di rotta (§6.6) tornano a consentire l'accesso pieno alle operazioni.

4 Wireframe descrittivi delle schermate

4.1 Notazione

Ogni schermata è descritta per regioni di layout (intestazione, corpo, aree laterali o modali) e per elementi ordinati dall'alto verso il basso. I riferimenti tra parentesi (UC/RF) individuano il requisito di origine; i riferimenti preceduti da **Componente:** rimandano al catalogo del Design System (§5.3), in cui l'elemento è definito e stilizzato una singola volta.

4.2 Registrazione — /register

Corpo. Modulo centrato a colonna singola:

- Campo testo **Nome** (UC1.1, RF.2).
- Campo testo **Cognome** (UC1.1, RF.2).
- Campo testo **Indirizzo email** (UC1.3, RF.3), con validazione inline al cambio del focus.
- Campo **Password** (UC1.4, RF.3).
- **Selettore di ruolo** (UC1.5, RF.4) a scelta singola (Sviluppatore, Security Auditor / Team Lead, Project Manager) con riga di descrizione sintetica per opzione.
- Pulsante di conferma primario **“Crea account”**.
- Collegamento secondario verso /login per l'utente già registrato.

Stati.

- *Email già registrata* (UC2/RF.5): messaggio non bloccante sopra il modulo, con link diretto a /login.
- *Dati non validi* (UC3/RF.6): **Componente:** Campo di modulo con validazione inline. Il pulsante primario resta disabilitato.

4.3 Login — /login

Corpo. Modulo centrato:

- Campo testo **Indirizzo email** (UC4.1, RF.8).
- Campo **Password** (UC4.2, RF.8).
- Pulsante di conferma primario **“Accedi”**.
- Collegamento secondario verso /register.

Stati.

- *Credenziali errate* (UC5/RF.9): messaggio d'errore generico posizionato sopra il modulo, senza indicare quale campo specifico è errato.
- *Successo*: redirect condizionale a `/credentials` (se token assente) o `/select` (se token presente e valido).

4.4 Servizi connessi — `/credentials`

Corpo. Lista generica di provider (nell'MVP, un solo elemento).

- Elemento **“GitHub”** con sottotitolo esplicativo. Come argomentato in §2.3, questo singolo campo collassa e assolve contestualmente all'inserimento del token per l'accesso al codice (RF.11) e per l'accesso al sistema di Task Management (RF.12).
- *Se non configurato*: campo di input per il token e pulsante **“Connetti”**.
- *Se configurato*: etichetta di stato **“Connesso”** con timestamp, pulsanti **“Verifica di nuovo”** e **“Rimuovi”**.
- Area informativa inferiore sulla predisposizione per futuri provider.

Stati.

- *Formato non valido* (UC8/RF.14): errore inline sul campo prima dell'invio.
- *Validazione fallita* (UC7/RF.13): messaggio d'errore del servizio. Pulsante **“Continua”** disabilitato.
- *Credenziale invalidata a runtime* (§2.3): messaggio dedicato **“Il token risulta scaduto o revocato”**, stesso comportamento dello stato di validazione fallita.

4.5 Selezione del contesto — `/select`

Corpo, prima area (Repository e riferimento).

- Campo testo **URL del repository Git** (UC9.1, RF.16).
- Campo testo **Nome del branch** (UC9.3, RF.17), con autocompletamento attivo post-validazione URL (UC11/RF.20).
- Campo testo opzionale **Hash del commit** (UC9.4, RF.17).

Corpo, seconda area (Ambito di analisi, abilitata post-validazione).

- **Componente:** **Albero file** per la selezione a tre stati (file/directory). Selezionare una directory marca ricorsivamente i figli (UC16.2, RF.28).
- Comando rapido **“Seleziona intero repository”** (UC16.1, RF.26).
- Contatore in tempo reale (file e righe stimate) con soglia visiva (UC19/RF.31).
- Pulsante di conferma primario **“Conferma contesto”**.

Stati.

- *Repository o riferimento non trovati/permessi mancanti*: errore inline sul campo.
- *Linguaggio non supportato*: avviso non bloccante sopra l'albero (UC15/RF.24).
- *Ambito vuoto/inaccessibile*: messaggio d'errore bloccante prima dell'invio (UC17, UC18).

4.6 Avvio operazioni — /run

Corpo.

- Lista delle operazioni disponibili, già filtrata in base al ruolo utente (§2.1). Ogni operazione della Tabella 2 — incluso `CHANGELOG_BUSINESS` — compare come voce indipendente e paritetica, coerentemente con UC20.1/UC21.1.
- Per ogni operazione: Nome, descrizione sintetica, casella di selezione.
- `CHANGELOG_BUSINESS` è selezionabile e avviabile autonomamente, senza necessità di selezionare anche `CHANGELOG_TECHNICAL`: la generazione tecnica è inclusa come primo passo interno della stessa Task (§3.6.1).
- Pulsante di conferma primario **“Avvia”**.

Stati.

- *Nessuna operazione selezionata*: impedimento all'attivazione del pulsante (UC21.4/RF.41).

4.7 Dashboard — /tasks

Corpo. Lista delle operazioni del batch corrente (UC22).

- Nome dell'operazione.
- **Componente:** **Indicatore di stato** (*In attesa, In elaborazione, Completato, Fallito, Annullato*) e barra di avanzamento (RF.46).
- Comando **“Annulla”** (attivo in *attesa/elaborazione*).
- Comando **“Visualizza report”** (attivo in *Completato* o *Fallito*).

Aree interattive (Agente Changelog). Espansioni *inline* per le task in stato di richiesta input:

- *Richiesta Sprint ID* (UC41.1): campo testo, comandi “Conferma/Annulla” (UC42).
- *Metadati insufficienti* (UC43): elenco identificativi scartabili, comandi “Procedi escludendo/Annulla operazione” (UC44).

- *Conferma Changelog di business* (`pendingInput.kind === 'BUSINESS_CONFIRMATION'`): anteprima del changelog tecnico appena generato, comandi “Accetta e genera Business” (UC45) / “Annulla” (UC46).

L'elemento della Dashboard associato a un'operazione `CHANGELOG_BUSINESS` resta un unico elemento per l'intera durata del flusso (generazione tecnica, attesa conferma, generazione business): non vengono mai mostrate due righe distinte per la stessa selezione.

Stati.

- *Errore di orchestrazione*: **Componente**: Stato d'errore limitato al singolo elemento fallito, garantendo l'isolamento visivo (UC23/RF.48).

4.8 Dettaglio report — `/reports/:id`

Intestazione. Riga di metadati: repository, riferimento, ambito analizzato, tempo di esecuzione, timestamp (UC26.1).

Corpo (Dispatcher a blocchi). Ordine e tipo definiti dalla risposta API:

- **TextBlock**: Markdown renderizzato (changelog).
- **FindingBlock / PolicyViolationBlock**: Liste strutturate filtrabili per gravità (Agente Security) con **Componente**: **Snippet di codice** per le remediation.
- **Proposal**: **Componente**: Collegamento Pull Request (elemento primario) e anteprima diff collapsabile (Agente Docs).
- **Modulo di validazione**: Comandi di accettazione/annullamento (changelog di business).

Comandi di pagina.

- Pulsante “**Esporta PDF**” (UC28/RF.73) con indicatore di caricamento.

Stati.

- *Status FAILED*: Corpo sostituito dal **Componente**: Stato d'errore (tipo, fase, messaggio).
- *Esportazione fallita*: Errore transitorio a pagina (UC29).

4.9 Storico report — `/reports`

Corpo.

- Lista di elementi report (UC24): identificativo, titolo descrittivo, timestamp.
- Selezione elemento → navigazione verso `/reports/:id` (UC25).

4.10 Intestazione applicativa

Elemento persistente su tutte le rotte autenticate.

- Nome dell'utente autenticato.
- **Componente:** Indicatore di stato utilizzato come badge di sola lettura per il ruolo assegnato.
- **Banner di credenziale invalidata** (§2.3): fascia d'avviso non bloccante per la navigazione in lettura, con collegamento a `/credentials`.
- Comando “**Esci**”.

Stati.

- *Fallimento di rete (Logout)*: Errore contestuale al tentativo di uscita.

5 Design System e componenti condivisi

5.1 Stack di implementazione

Il design system dell'MVP è implementato utilizzando **Tailwind CSS** come base per le classi di utilità (utility-first) e **shadcn/ui** come libreria di primitive accessibili per i componenti interattivi complessi (modali, selettori, form). Questa scelta tecnologica garantisce nativamente il rispetto del requisito **RV.5**, assicurando la piena compatibilità senza anomalie grafiche sulle versioni più recenti dei principali browser desktop (Google Chrome, Mozilla Firefox, Microsoft Edge v145 o superiore).

5.2 Token visivi

I token sono definiti come variabili di tema globali, in modo da poter essere declinati visivamente senza modificare il codice dei singoli componenti che li consumano.

Categoria di token	Uso previsto
Colori semantici di stato	Un colore per ciascuno dei cinque stati operativi (<i>Attesa, Elaborazione, Completato, Fallito, Annullato</i>). Riutilizzati identicamente nell' Indicatore di stato e in ogni badge o messaggio che comunica un esito.
Colori semantici di gravità	Un colore per ciascun livello di gravità delle segnalazioni dell'Agente Security (<i>Critica, Alta, Media, Bassa, Informativa</i>). Il livello informativo è riutilizzato per gli avvisi di complessità dell'Agente Docs.
Colore di superficie neutra	Sfondo di pagina, delle card e dei moduli; un'unica scala di grigi utilizzata per l'intera applicazione.
Tipografia	Una famiglia font per il testo dell'interfaccia (etichette, corpo, moduli) e una famiglia monospaziata dedicata esclusivamente agli Snippet di codice e ai riferimenti a percorsi/righe di file.

Categoria di token	Uso previsto
Spaziatura	Scala unica su base 4px, applicata uniformemente per garantire coerenza tra densità informative molto diverse (es. form di login vs lista report Security).
Raggio e ombreggiatura	Valori unici per card, finestre modali e badge, per mantenere la stessa grammatica visiva tra le diverse aree dell'applicativo.

Tabella 4: Categorie di token visivi condivisi e ambito di applicazione.

5.3 Catalogo dei componenti condivisi

I seguenti componenti sono definiti una sola volta e riutilizzati in tutte le schermate descritte in §4.

- **Indicatore di stato:** Etichetta testuale con colore semantico associato, usata nella Dashboard e per lo stato dei servizi connessi. Include opzionalmente una barra di avanzamento.
- **Badge di gravità:** Etichetta compatta con colore semantico, usata per i riscontri di sicurezza e gli avvisi di complessità del codice.
- **Albero file:** Struttura ad albero navigabile con caselle di selezione a tre stati (non selezionato, selezionato, parzialmente selezionato), usata nella selezione dell'ambito di analisi.
- **Blocco di report:** Componente *dispatcher* che, dato un elemento del payload di risposta, renderizza il sottocomponente corretto in base al tipo (testo, lista, snippet, modulo).
- **Collegamento Pull Request:** Pulsante o link in evidenza verso la Pull Request aperta dal backend, con anteprima del *diff* disponibile come dettaglio secondario collapsabile.
- **Snippet di codice:** Contenitore monospaziato con evidenziazione sintattica minima, usato per il codice correttivo proposto e per le anteprime diff.
- **Modulo di validazione:** Coppia di comandi (primario/secondario) con testo esplicativo, usato per innescare i changelog di business e per le aree interattive della Dashboard.
- **Stato d'errore:** Componente a pagina intera o a riquadro che riporta il tipo di errore, la fase in cui si è verificato e il messaggio descrittivo (§8).
- **Campo di modulo con validazione inline:** Campo di input con stato neutro/errore e messaggio contestuale posizionato al di sotto del campo stesso.

5.4 Stati interattivi comuni

Ogni componente che avvia una richiesta di rete (REST) espone sempre tre stati distinti, per garantire un feedback visivo immediato:

1. **In corso:** Il controllo che ha originato la richiesta mostra un indicatore di caricamento e viene disabilitato per prevenire invii duplicati.
2. **Esito positivo:** Transizione visiva immediata verso lo stato successivo previsto (es. redirect, aggiornamento lista, chiusura modale). L'utente non viene mai lasciato sulla schermata originaria con un semplice messaggio di conferma senza indicazioni.
3. **Esito negativo:** Attivazione del componente **Stato d'errore** o del campo in stato di errore (§8), senza perdita dei dati già inseriti dall'utente nella form.

5.5 Accessibilità (A11y)

L'utilizzo di `shadcn/ui` fornisce nativamente la navigazione da tastiera, la corretta attribuzione dei ruoli ARIA e la gestione visibile del focus. Come regola aggiuntiva di progettazione, i colori semantici di stato e gravità (§5.2) devono essere sempre accompagnati da un'etichetta testuale esplicita, e non essere mai affidati alla sola percezione cromatica.

6 Stato applicativo

6.1 Principio generale

L'applicazione adotta una politica rigorosa per la gestione dello stato globale: entra in uno *store* Zustand esclusivamente ciò che deve sopravvivere a un cambio di rotta (pagina) o che deve essere letto contemporaneamente da più componenti distanti nell'albero. Tutto il resto rimane relegato allo stato locale del singolo componente o della singola pagina.

La Figura 3 illustra l'architettura dei tre store globali dell'applicativo e le loro direttrici di comunicazione.

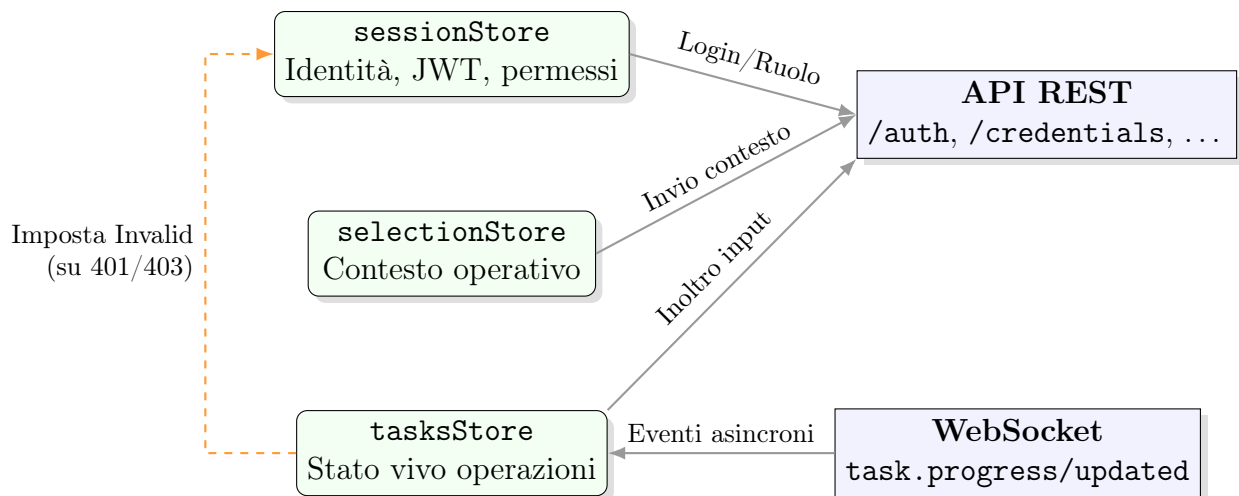


Figura 3: Architettura degli store applicativi, comunicazione di rete ed effetti collaterali.

Attenzione

Il JSON Web Token non deve **mai** essere salvato all'interno del `localStorage` o del `sessionStorage` del browser. Poiché l'MVP non prevede protezioni WAF avanzate contro attacchi XSS, conservare il token esclusivamente nella memoria volatile dello store Zustand garantisce che uno script malevolo non possa estrarlo in modo persistente.

6.2 sessionStore — Identità e credenziali

Mantiene la sessione utente e lo stato autorizzativo. Il JSON Web Token (JWT) è mantenuto esclusivamente in memoria (nello store) e mai persistito nel `localStorage`. Una ricarica completa della pagina richiede un nuovo login.

```

interface SessionState {
  user: { id: string; firstName: string; role: UserRole } | null
  token: string | null
  credentialsStatus: 'unknown' | 'missing' | 'connected' | 'invalid'
  isAuthenticated: () => boolean

  login(user, token): void
  logout(): void
  setCredentialsStatus(status): void
  markCredentialsInvalid(): void
}

type UserRole = 'DEVELOPER' | 'SECURITY_AUDITOR' | 'PROJECT_MANAGER'
  
```

Listing 1: Interfaccia del sessionStore.

- **Campo role:** Unica fonte di verità per le regole di visibilità delle operazioni (§2.1). Popolato al login e immutabile per la durata della sessione.

- **Campo `credentialsStatus`:** Evita di interrogare `GET /credentials` su ogni rotta protetta. Il valore `'invalid'` differisce da `'missing'`: indica che una credenziale configurata è stata revocata o è scaduta durante l'utilizzo.

6.3 `selectionStore` — Contesto operativo

Contiene il `contextId` e i parametri dell'ambito di analisi in uso nella sessione corrente. Viene popolato dalla pagina `/select` e letto in fase di avvio dalle richieste della pagina `/run`.

6.4 `tasksStore` — Stato vivo delle operazioni

Contiene lo stato in tempo reale delle operazioni correnti, aggiornato asincronamente dagli eventi WebSocket inviati dal backend. La Figura 4 descrive la macchina a stati di una Task gestita da questo store.

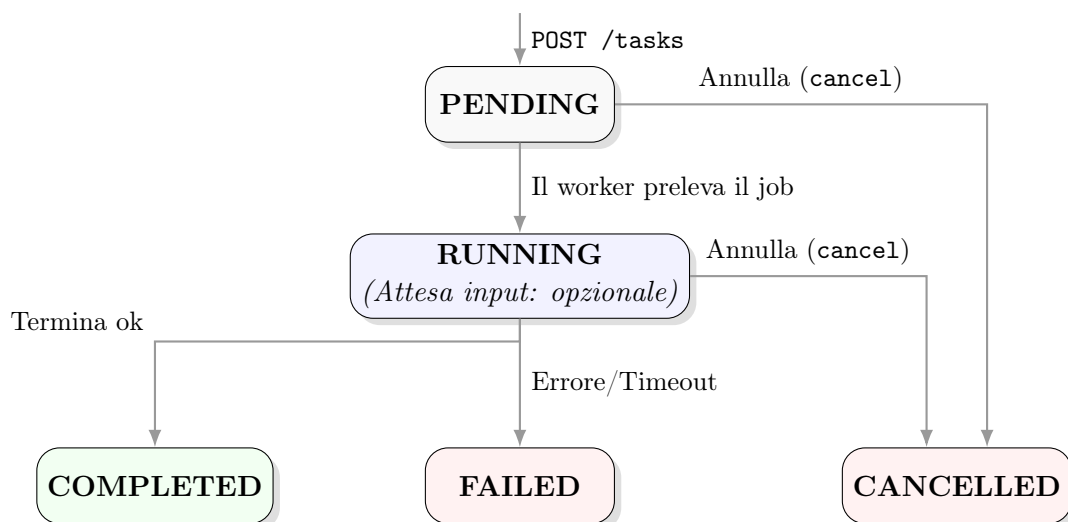


Figura 4: Ciclo di vita di una Task. La richiesta di input (`pendingInput`) non costituisce un nuovo stato, ma una condizione sospensiva interna allo stato `RUNNING`.

Il campo `pendingInput`, quando valorizzato, è il segnale che innesca l'espansione dell'area interattiva sulla Dashboard (§4). L'annullamento della Task, anche durante la fase di input, avviene invocando l'azione unificata `cancel(taskId)`.

```

interface TasksState {
  tasks: Record<string, TaskEntry>

  upsertFromEvent(event: TaskEvent): void // event: task.progress | task.
    updated
  cancel(taskId): void // chiamata a POST /tasks/:id/cancel
  resolvePendingInput(taskId, response): void // risoluzione input utente
}

interface TaskEntry {

```

```

    id: string
    operation: OperationCode
    status: 'PENDING' | 'RUNNING' | 'COMPLETED' | 'FAILED' | 'CANCELLED'
    progressPercent: number
    currentStage: string | null
    reportId: string | null
    error: { code: string; message: string; stage: string } | null

    pendingInput: PendingInput | null
}

type PendingInput =
  | { kind: 'SPRINT_ID' }
  | { kind: 'INCOMPLETE_TASKS'; taskIds: string[] }
  | { kind: 'BUSINESS_CONFIRMATION'; technicalReportId: string }

```

Listing 2: Interfaccia del tasksStore con gestione input interattivi.

Effetto collaterale su sessionStore: Alla ricezione dell'evento `task.failed` con `error.code === 'CREDENTIAL_INVALID'`, l'handler del `tasksStore` invoca l'azione `markCredentialsInvalid()` del `sessionStore`. È l'unico punto architetturale in cui uno store modifica lo stato di un altro, in risposta a un evento di rete che invalida le credenziali a livello globale.

6.5 Dati intenzionalmente non globali

Per il principio di separazione dichiarato, le seguenti entità rimangono confinate allo stato locale di React (`useState` o `useQuery`):

- La lista dei servizi connessi (GET `/credentials`), letta e mostrata unicamente dalla pagina `/credentials`.
- L'albero dei file del repository e l'elenco di *refs*, necessari solo in `/select`.
- La cache dello storico dei report (GET `/reports`), mantenuta per la sola pagina `/reports`.
- Il contenuto del singolo report aperto (GET `/reports/:id`).

6.6 Guardie di rotta

Le restrizioni di navigazione client-side si basano unicamente sulla lettura sincrona del `sessionStore`. Non vengono effettuate chiamate di rete al momento del cambio rotta.

- **Accesso bloccato per utenti autenticati:** `/login` e `/register` (redirect a `/run`).
- **Accesso bloccato per utenti non autenticati:** Tutte le altre rotte (redirect a `/login`).

- **Accesso in lettura a Task e Report:** Libero per tutti gli utenti autenticati, indipendentemente dallo stato delle credenziali GitHub (inclusi 'missing' o 'invalid').
- **Avvio nuove operazioni bloccato:** Se `credentialsStatus` è 'missing' o 'invalid', l'accesso a `/select` e la conferma di avvio su `/run` effettuano un redirect immediato a `/credentials`.

7 Contratto API consumato

7.1 Principi generali del contratto

Il contratto API si appoggia sul design architetturale già validato nella fase di Proof of Concept. Il frontend aderisce ai seguenti vincoli strutturali:

- **Prefisso globale:** Tutte le rotte consumate dal frontend sono esposte sotto il prefisso `/api/v1`. Le chiamate utilizzano esclusivamente percorsi relativi, delegando l'instradamento all'infrastruttura sottostante (dettagliato in §13).
- **Separazione dei piani:** Esiste una netta separazione tra il piano pubblico (browser → backend) e il piano interno (agenti → backend). Il frontend ha visibilità e accesso unicamente al piano pubblico.
- **Autenticazione realtime:** La connessione al canale WebSocket viene autenticata trasmettendo il JSON Web Token (JWT) durante la fase di *handshake*.

Attenzione

Vincoli di Rete Ereditati dall'Infrastruttura AWS:

- **Nessun setup CORS:** Il frontend e il backend sono serviti dalla stessa origine (CloudFront). Non configurare intercettori per preflight OPTIONS, né iniettare header CORS custom nei client HTTP (come Axios o Fetch API).
- **Path Relativi (Vite):** È categoricamente vietato inserire URL assoluti (es. `https://api...`) all'interno della variabile d'ambiente `VITE_API_BASE_URL`. Usare sempre percorsi relativi (es. `/api/v1`). Hardcodare un dominio causerà il fallimento silente delle pipeline CI/CD tra i vari ambienti.

7.2 Endpoint REST pubblici

La Tabella 5 elenca gli endpoint del piano pubblico consumati dal frontend, inclusi delle estensioni implementate per l'MVP (autenticazione, gestione credenziali, storico, esportazione PDF ed elaborazione input asincroni).

L'endpoint di esportazione report utilizza esclusivamente il formato PDF. La generazione avviene lato backend e il frontend consuma il file direttamente in streaming (nel corpo della risposta HTTP).

Metodo e Path	Scopo	Requisiti
Autenticazione		
POST /auth/register	Registrazione, include il ruolo operativo	RF.1–6
POST /auth/login	Login, restituisce il JWT	RF.7–9
GET /auth/me	Profilo e ruolo dell'utente autenticato	RF.7
Servizi Connessi		
GET /credentials	Elenco dei provider configurati e relativo stato	RF.10
POST /credentials	Salva una credenziale per un provider	RF.10–11
POST /credentials/:id/validate	Verifica la validità e i permessi del token	RF.13, RS.4
DELETE /credentials/:id	Rimuove una credenziale	UC6
Contesto Operativo		
GET /repositories	Elenco dei repository accessibili dall'utente	RF.15
GET /repositories/:owner/:repo/refs	Branch disponibili	RF.16–17
GET /repositories/:owner/:repo/tree	Struttura del repository (richiede ?ref=)	RF.25–28
POST /contexts	Crea l'AnalysisContext	RF.15–31
Avvio ed Esecuzione		
GET /operations	Operazioni disponibili (già filtrate per ruolo)	RF.32–34
POST /tasks	Avvia una o più task	RF.36–42
GET /tasks	Elenco delle task per la Dashboard	RF.43–45
GET /tasks/:id	Dettaglio di una singola task	RF.45–46
POST /tasks/:id/cancel	Annulla una task, anche durante l'attesa di input	RF.46, UC42, UC44
POST /tasks/:id/input	Risolve una richiesta di input utente in sospenso (Sprint ID o esclusione task)	UC41.1, UC43, RF.98
Storico e Report		
GET /reports	Storico dei report	RF.49–52
GET /reports/:id	Dettaglio di un report	RF.53–64
GET /reports/:id/export?format=pdf	Esporta il report in PDF (streaming binario)	RF.73–74

Tabella 5: Endpoint REST pubblici consumati dal frontend, raggruppati per area funzionale.

7.3 Canale realtime (WebSocket)

Il canale WebSocket è utilizzato in modalità unidirezionale (backend → frontend) per il flusso applicativo. Il frontend emette unicamente l'evento di sottoscrizione, mentre le risposte applicative fornite dall'utente non transitano per il socket, bensì tramite l'endpoint REST POST `/tasks/:id/input`.

In caso di caduta della connessione di rete, alla riapertura del socket il frontend esegue un polling di riallineamento tramite GET `/tasks` per recuperare l'eventuale stato `pendingInput` o gli avanzamenti persi.

Attenzione

I meccanismi di riconnessione automatica integrati nel client Socket.IO non recuperano i messaggi persi durante il periodo di disconnessione. È responsabilità del frontend, all'evento di `reconnect`, invocare l'endpoint REST GET `/tasks` per allineare forzatamente lo stato del `tasksStore`, recuperando eventuali variazioni di percentuale o richieste di input sfuggite al socket.

Evento	Payload	Uso nel frontend
<code>task.updated</code>	<code>{ taskId, status, reportId? }</code>	Aggiorna l'indicatore di stato nella Dashboard.
<code>task.progress</code>	<code>{ taskId, stage, percent }</code>	Avanza la barra di progresso.
<code>task.failed</code>	<code>{ taskId, error }</code>	Attiva lo stato di errore isolato sulla singola task (§8). Il codice <code>'CREDENTIAL_INVALID'</code> ha effetti globali.
<code>batch.completed</code>	<code>{ batchId, completed, failed }</code>	Sblocca i controlli successivi (es. abilitazione link Report).
<code>task.inputRequired</code>	<code>{ taskId, kind, taskIds?, reportId? }</code>	Valorizza <code>pendingInput</code> nel <code>tasksStore</code> (§6.4), forzando l'espansione dell'area interattiva. Il campo <code>reportId</code> è presente quando <code>kind === 'BUSINESS_CONFIRMATION'</code> (§3.6.1), per mostrare l'anteprima del changelog tecnico appena generato.

Tabella 6: Eventi WebSocket emessi dal backend verso il frontend.

7.4 Schemi dei DTO principali

Gli schemi (Data Transfer Object) implementati nel frontend riflettono esattamente il contratto TypeScript fornito dall'API. L'enumerazione `OperationCode` ricalca le operazioni definite in §2.1.

```
// --- Body di POST /auth/register
interface RegisterDto {
  firstName: string
  lastName: string
  email: string
  password: string
  role: 'DEVELOPER' | 'SECURITY_AUDITOR' | 'PROJECT_MANAGER'
}

// --- Risposta di POST /auth/login e POST /auth/register
interface AuthResponseDto {
  token: string
  user: { id: string; firstName: string; role: string }
}

// --- Credenziali di Servizio ---
interface ServiceCredentialDto {
  id: string
  provider: 'GITHUB'
  status: 'CONNECTED' | 'INVALID'
  lastValidatedAt: string | null
}

// --- Struttura di una Task e degli Input Asincroni ---
interface TaskDto {
  id: string
  batchId: string
  operation: OperationCode
  status: 'PENDING' | 'RUNNING' | 'COMPLETED' | 'FAILED' | 'CANCELLED'
  progressPercent: number
  currentStage: string | null
  reportId: string | null
  error: { code: string; message: string; stage: string } | null
  pendingInput: {
    kind: 'SPRINT_ID' | 'INCOMPLETE_TASKS' | 'BUSINESS_CONFIRMATION'
    taskIds?: string[]
    technicalReportId?: string
  } | null
}

// --- Body di POST /tasks/:id/input ---
type SubmitInputDto =
  | { kind: 'SPRINT_ID'; sprintId: string }
  | { kind: 'INCOMPLETE_TASKS'; action: 'PROCEED' | 'CANCEL' }
```

```
| { kind: 'BUSINESS_CONFIRMATION'; action: 'PROCEED' | 'CANCEL' }

// --- Elemento restituito da GET /reports ---
interface ReportSummaryDto {
  id: string
  operation: OperationCode
  status: 'COMPLETED' | 'FAILED'
  generatedAt: string
  title: string
}
```

Listing 3: Schemi dei DTO necessari al frontend per l'MVP.

7.5 API interne

Le API interne (`/internal/*`) sono adibite alla sola comunicazione machine-to-machine tra gli agenti intelligenti e il backend applicativo. Sono autenticate tramite firme HMAC. Il frontend non le consuma, non vi ha accesso di rete e non è tenuto a integrarne le interfacce nel proprio perimetro di progetto.

Gestione degli errori

8.1 Due famiglie di errore

Il frontend gestisce due categorie distinte di errori, trattate con due pattern di interfaccia differenti:

- **Errori di validazione e configurazione:** Sincroni. Emergono durante la compilazione di un modulo o la conferma di una selezione. Vengono risolti immediatamente dall'utente rimanendo sulla stessa schermata.
- **Errori di esecuzione di un'operazione:** Asincroni. Emergono a distanza di tempo dall'avvio di un'operazione, solitamente notificati tramite canale realtime o al caricamento della Dashboard.

8.2 Pattern per gli errori di validazione e configurazione

La regola generale prevede che il messaggio di errore compaia *contestualmente* (al di sotto del campo o dell'area interessata), avvalendosi del **Componente: Campo di modulo con validazione inline**.

Eccezioni e regole di comportamento:

- **Login (Credenziali errate):** Il messaggio è generico e posizionato sopra il modulo, senza indicare quale campo specifico (email o password) sia errato.
- **Persistenza dati:** I dati già inseriti dall'utente restano sempre visibili e non vengono mai azzerati a seguito di un errore di validazione.

- **Blocco rete:** Nessuna chiamata di rete aggiuntiva verso le API o verso servizi esterni (es. validazione token GitHub) viene effettuata finché l'errore locale non è stato corretto.

8.3 Pattern per gli errori di esecuzione di un'operazione

Gli errori che si verificano durante l'esecuzione dell'agente o dell'orchestrazione vengono propagati all'interfaccia seguendo questo flusso:

1. Il backend porta lo stato della task a **FAILED** e lo notifica via WebSocket.
2. Nella Dashboard, esclusivamente l'elemento interessato passa allo stato "Fallito", garantendo l'isolamento (le altre operazioni continuano ad aggiornarsi senza interruzione).
3. L'elemento fallito espone il comando "Visualizza report". L'apertura della vista dettaglio sostituisce i blocchi di contenuto con il **Componente: Stato d'errore**.

Il componente **Stato d'errore** è parametrizzato per mostrare il messaggio specifico, la fase di interruzione e un'eventuale azione secondaria, come mappato nella Tabella 7.

Causa / Trigger	Messaggio in interfaccia	Azione secondaria
Superamento soglia massima di utilizzo LLM	Limite di utilizzo del modello AI raggiunto; riprovare successivamente.	Nessuna (blocco temporizzato)
Timeout del modello LLM (oltre soglia configurata)	Il modello AI ha impiegato troppo tempo a rispondere.	"Riprova" (riavvia sola task)
Output del modello LLM non parsabile	L'output generato dall'AI non è nel formato atteso.	"Riprova"
Superamento dei limiti dimensionali a runtime	Il contesto analizzato eccede la capacità di lettura del modello.	Rimanda a <code>/select</code> per ridurre l'ambito
Risorsa di contesto mancante (es. <code>POLICY.md</code> , Sprint ID)	Risorsa richiesta non trovata: <i>[nome risorsa]</i> .	Nessuna
Risorsa di contesto malformata	La risorsa richiesta non è interpretabile: <i>[dettaglio problema]</i> .	Nessuna
Fallimento creazione Pull Request (permessi insufficienti)	Impossibile aprire la Pull Request; verificare i permessi del token.	Rimanda a <code>/credentials</code>
Token GitHub scaduto o revocato a runtime	Il token configurato non è più valido; verificalo su "Servizi connessi".	Rimanda a <code>/credentials</code>

Tabella 7: Mappatura degli errori di esecuzione sul componente **Stato d'errore**.

8.3.1 Errore di orchestrazione o routing

I fallimenti dovuti a disservizi infrastrutturali o di rete che non derivano esplicitamente dalla logica interna degli agenti o dalle validazioni (es. caduta connettività verso GitHub) vengono gestiti dal frontend come errori generici. Il messaggio mostrato è: *“Impossibile completare l’operazione per un errore interno di sistema”*. L’unica azione secondaria proposta è il comando generale “Riprova”.

8.4 Errore di esportazione (Report)

Il fallimento dell’esportazione PDF lato backend diverge dal pattern di esecuzione: l’errore viene mostrato come messaggio transitorio (toast/alert) nella pagina di dettaglio del report. Non altera lo stato del report (che rimane **COMPLETED**) e non sostituisce i blocchi di contenuto renderizzati a schermo, permettendo all’utente di continuare la consultazione.

9 Tracciamento Requisito → Decisione Frontend

Questa sezione mappa i requisiti funzionali e qualitativi (definiti nell’Analisi dei Requisiti) sulle scelte operative e implementative applicate nel frontend. Al fine di mantenere la tabella focalizzata sulle scelte architetturali, non sono elencati i requisiti funzionali di base (relativi alla mera presenza di campi di testo o pulsanti) o i requisiti fuori dal perimetro di competenza del client.

9.1 Decisioni Operative e Architetture

Requisito	Descrizione Sintetica	Implementazione Frontend	Rif.
RQ.10	Tempo di risposta percepito ≤ 2 secondi	Adozione di sessionStore in memoria per guardie di rotta sincrone (zero chiamate bloccanti al cambio pagina) ed esposizione nativa degli stati di caricamento per ogni componente interattivo.	§5.4, §6.6
RQ.11	Apprendibilità sistema ≤ 30 minuti	Filtraggio visivo a monte: la pagina di avvio operazioni nasconde totalmente le opzioni disabilitate per il ruolo attivo, abbassando il carico cognitivo.	§2.1
RF.4	Scelta del ruolo operativo	Componente: Selettore di ruolo integrato esclusivamente nella fase di registrazione, definendo il ruolo per l’intera vita dell’account.	§4

Requisito	Descrizione Sintetica	Implementazione Frontend	Rif.
RF.14, RS.4	Validazione sintattica e logica token	Componente di validazione <i>inline</i> sul client per la sintassi, seguito da validazione esplicita via API per garantire autorizzazioni remote sufficienti prima del salvataggio.	§8.2
RF.36, RF.37	Selezione e avvio multiplo	Adozione di una lista con caselle di controllo indipendenti in <code>/run</code> e invio batch tramite array nel payload della <code>POST /tasks</code> .	§3.3
RF.40, UC41, UC45, UC46	Dipendenza Changelog tecnico → business	<code>CHANGELOG_BUSINESS</code> è selezionabile in <code>/run</code> come operazione indipendente e autonoma (UC21.1, UC41), senza richiedere la co-selezione di <code>CHANGELOG_TECHNICAL</code> . L'Orchestratore genera il changelog tecnico come primo passo interno della stessa Task e la sospende (<code>pendingInput.kind === 'BUSINESS_CONFIRMATION'</code>) finché l'utente non conferma esplicitamente dal report intermedio (UC26.2.5), riusando il medesimo meccanismo già adottato per lo Sprint ID.	§3.6.1, §6.4
RF.46	Avanzamento stato esecuzione	Sottoscrizione WebSocket agli eventi <code>task.progress</code> e <code>task.updated</code> , aggiornamento reattivo nella Dashboard tramite <code>tasksStore</code> .	§6.4
RF.48	Isolamento dei fallimenti in esecuzione	L'errore di una singola Task attiva il Componente: Stato d'errore limitato al solo elemento in Dashboard, le altre Task proseguono l'aggiornamento grafico.	§8.3
RF.98– RF.101	Richiesta input umano a metà esecuzione	Gestione dello stato <code>pendingInput</code> interno allo stato <code>RUNNING</code> ; l'UI espande un'area modale <i>inline</i> direttamente sull'elemento della lista. In caso di taglio budget, l'interfaccia non ostacola un approccio automatizzato <i>stub</i> .	§3.6, §12
RF.73	Esportazione report PDF	Consumo del file tramite streaming binario su rete same-origin senza redirect esterni; indicatore di caricamento globale per la durata del download.	§7

Requisito	Descrizione Sintetica	Implementazione Frontend	Rif.
RF.63, RS.3	Divieto scrittura autonoma e accesso PR	Coerentemente al vincolo RS.3, il frontend non espone comandi di approvazione interna o "Push" verso Git: sostituisce la <i>diff view</i> primaria con un Collegamento Pull Request delegando l'azione umana alla piattaforma GitHub.	§5.3
RS.2	Token e credenziali protetti	Il JWT di sessione non viene mai scritto nel <code>localStorage</code> per mitigare l'estrazione via attacchi XSS; conservazione in sola memoria volatile.	§6.2

Tabella 8: Mappatura Requisiti - Decisioni operative del Frontend.

9.2 Requisiti mappati a livello di Flussi e Wireframe

Per evitare estrema ridondanza documentale, i requisiti puramente funzionali relativi alla presenza di specifici campi di input, pulsanti, visualizzazione formattata dei report o logiche standard di routing (es. **RF.1–RF.3**, **RF.15–RF.35**, **RF.49–RF.64**) non sono elencati singolarmente in questa tabella. La loro soddisfazione è garantita strutturalmente e visivamente all'interno dei **Flussi Utente** (§3) e dei **Wireframe descrittivi** (§4), in cui ogni elemento grafico o schermata cita esplicitamente il requisito funzionale (RF) e il caso d'uso (UC) di origine.

9.3 Requisiti Fuori Perimetro Frontend

I seguenti requisiti non compaiono nel tracciamento poiché la loro soddisfazione dipende interamente dall'infrastruttura AWS, dal backend Node.js o dal core degli agenti Python, e non richiedono alcuna implementazione tecnica specifica nel codice sorgente del Frontend:

- **RV.1**, **RV.10–RV.15**, **RV.21**: Vincoli tecnologici e architetturali di backend, database, agenti e infrastruttura AWS.
- **RF.82–RF.97**, **RF.104**: Logica di business per l'individuazione di vulnerabilità, documentazione del codice, scraping dei ticket e traduzione da parte dei modelli LLM.
- **RS.1**, **RS.5**: Sicurezza crittografica degli hash delle password nel database e isolamento e data-residency sul modello LLM.
- **RQ.1–RQ.8**: Copertura test di codice sorgente backend/agenti, complessità ciclica, efficienza e accuratezza del modello AI e metriche Flesch Reading Ease.

Parte II

Motivazioni e Decisioni Architettureali

Questa sezione raccoglie le argomentazioni, i principi guida e le decisioni architetturali (ADR-FE) che hanno portato alla definizione delle interfacce, degli stati e dei flussi descritti nella Parte I.

10 Principi Guida e Vincoli di Progetto

Le scelte architettureali e di design del frontend per questo MVP sono state guidate da un set di vincoli applicati trasversalmente all'intero documento. Invece di ribadire queste giustificazioni in ogni singola schermata o flusso, vengono dichiarate qui come fondamento delle decisioni successive.

- **Vincolo Tempo/Risorse (Pragmatismo e Riuso):** Il team di sviluppo è composto da sei sviluppatori non a regime pieno, con una finestra di tempo di meno di due settimane per l'implementazione dell'MVP. Questo vincolo ha imposto il riuso sistematico di ogni componente, scelta di libreria e pattern architettonico già validato nella progettazione del Proof of Concept (stack React, Vite, TanStack Router, Zustand), estendendo e riprogettando solo le parti scritte originariamente per validare l'integrazione tecnologica ma non allineate ai requisiti.
- **Feedback immediato e reattività (RQ.10):** Il tempo di risposta percepito dell'interfaccia non deve superare i 2 secondi tra l'azione dell'utente e il feedback visivo. Poiché diverse operazioni dipendono da chiamate asincrone o tempi di latenza di rete (es. attesa dell'avvio degli agenti o esportazione PDF in streaming dal backend), il frontend deve sempre fornire indicatori di stato (es. *In corso*, *Esito positivo/negativo*) senza mai bloccare l'interfaccia in attesa della risoluzione.
- **Coerenza sopra varietà (RQ.11):** Un nuovo utente deve poter apprendere il sistema in 30 minuti o meno. Si predilige un numero ridotto di componenti condivisi e riusati ovunque (ad esempio, un unico componente per gli stati d'errore o per i moduli di validazione inline) rispetto a varianti specifiche create per singola schermata. Questo riduce drasticamente sia il carico cognitivo per l'utente, sia i tempi di sviluppo.
- **Interfaccia Desktop-First:** Code Guardian è uno strumento concepito per essere utilizzato da Sviluppatori, Security Auditor e Project Manager da postazione fissa durante il normale flusso di lavoro. L'Analisi dei Requisiti non pone alcun vincolo di supporto mobile; pertanto, il layout è progettato esclusivamente per viewport desktop, evitando di disperdere il budget di tempo nello sviluppo di versioni *responsive* non richieste per l'MVP.

Per offrire una visione immediata di come questi principi si traducano in scelte pratiche, la Tabella 9 mappa i vincoli principali sulle conseguenti direttive di progettazione del frontend.

Vincolo / Principio	Impatto Architettureale e UX
Tempi di sviluppo ristretti (Team ridotto)	Adozione di librerie pronte per le primitive di stile (Tailwind CSS, shadcn/ui) anziché creare un design system da zero. Adozione totale del contratto API REST e WebSocket già stabilito dal PoC.
Natura desktop dello strumento	Layout progettato a pagina intera, ottimizzando la densità informativa per le liste strutturate (es. report Agente Security) e l'albero dei file, senza logiche di impaginazione mobile.
Reattività (Soglia 2 secondi, RQ.10)	Nessuna rotta protetta effettua chiamate di rete bloccanti per verificare i permessi durante la navigazione: lo stato di autenticazione e ruolo è letto in modo sincrono da un <code>sessionStore</code> in memoria.
Semplicità di apprendimento (RQ.11)	Filtraggio a monte delle azioni: l'utente vede esclusivamente le operazioni che il suo ruolo operativo lo abilita a eseguire, eliminando la frustrazione visiva di elementi permanentemente disabilitati.

Tabella 9: Sintesi dei vincoli di progetto e relativo impatto sulla progettazione Frontend.

11 Decisioni Architettureali (ADR-FE) e Motivazioni

Di seguito vengono espansate e contestualizzate le decisioni architettureali cardine del frontend (ADR-FE), raggruppate per dominio di competenza. L'inventariazione di queste decisioni chiarisce il razionale dietro le specifiche descritte nella Parte I.

11.1 Identità, Sessione e Ruoli

- **ADR-FE-3 — Filtraggio delle operazioni disponibili per ruolo:** La schermata di avvio mostra esclusivamente le operazioni permesse al ruolo attivo dell'utente, omettendo del tutto quelle disabilitate. Un elenco pre-filtrato riduce il carico cognitivo, azzerando il rischio di interazioni non valide e soddisfa il requisito di apprendibilità (RQ.11). La sicurezza resta comunque garantita dal backend, che verifica il token a ogni richiesta.
- **ADR-FE-6 — Persistenza del JWT in sola memoria:** Il token di sessione viene conservato esclusivamente nello stato dell'applicazione (`sessionStore`) e mai nel `localStorage` o in cookie non-`httpOnly`. Questo minimizza la superficie di attacco XSS. Si accetta il compromesso UX che una ricarica completa della pagina (*F5*) comporti la perdita della sessione e richieda un nuovo login. Questa scelta è complementare, ma indipendente, dal rischio accettato lato infrastruttura sul trasporto in chiaro tra CloudFront e ALB.

- **ADR-FE-7 — Changelog di business come Task singola con checkpoint interno:** Coerentemente con la preconditione di UC41 (selezione di “changelog tecnico” o “changelog di business”), il frontend tratta `CHANGELOG_BUSINESS` come un’unica Task che incorpora al proprio interno la generazione tecnica come fase preliminare, riusando il medesimo meccanismo di sospensione già adottato per l’inserimento dello Sprint ID (`pendingInput`, §6.4). Questa scelta evita di introdurre un secondo concetto di “Task bloccante” nel `tasksStore` e mantiene un solo elemento in Dashboard per l’intero ciclo di vita dell’operazione, in linea con il principio di coerenza sopra varietà (§10).

11.2 Integrazioni e Credenziali

- **ADR-FE-4 — Credenziale Task Management unificata con GitHub:** L’ADR modella concettualmente due credenziali distinte (codice e issue). Tuttavia, poiché nell’MVP il Sistema di Task Management coincide con GitHub Issues, si è deciso di non richiedere all’utente di inserire due volte lo stesso token. La schermata dei Servizi Connessi è stata progettata come una lista generica (con un solo elemento “GitHub” per ora) affinché l’aggiunta futura di un provider diverso (es. Jira) richieda solo l’inserimento di un nuovo elemento nella lista, senza alcuna riprogettazione della pagina o del database.

11.3 Stack, Design System e Report

- **ADR-FE-5 — Adozione di Tailwind CSS e shadcn/ui:** In assenza di un design system preesistente, la necessità di sviluppare l’intera interfaccia in meno di due settimane ha reso impraticabile la scrittura di stili CSS personalizzati da zero. Tailwind CSS garantisce coerenza e velocità tramite classi di utilità, mentre shadcn/ui fornisce primitive accessibili (focus, ruoli ARIA, navigazione da tastiera) già pronte. La scelta è puramente di tempo-su-mercato (Time-to-Market) e non vincola la futura identità visiva del prodotto, essendo i token incapsulati in variabili di tema.
- **ADR-FE-2 — Il blocco Proposal privilegia il link alla Pull Request:** Nel PoC, la proposta dell’Agente Docs era visualizzata solo come un blocco diff isolato. Coerentemente con le decisioni di Progettazione AWS (e nello specifico con **ADR-AWS-10**, che delega al backend l’apertura automatica della PR per rispettare il vincolo RS.3), il frontend dell’MVP promuove il link alla Pull Request come elemento primario dell’interfaccia. L’anteprima del diff rimane disponibile come elemento secondario collassabile. Questo ribadisce che l’applicazione effettiva della modifica al repository resta un atto umano condotto esternamente al sistema, su GitHub.

12 Compromessi e Rischi Accettati

Il rispetto del vincolo temporale per lo sviluppo dell’MVP ha reso necessario bilanciare la completezza della *User Experience* (UX) con il pragmatismo del rilascio rapido. La Tabella 10 consolida i compromessi accettati a livello di frontend.

Risorsa / Scelta	Compromesso UX/Architettuale	Rischio Accettato
Design System Desktop-First	Nessuna media query o layout alternativo è stato implementato per gli schermi degli smartphone. L'interfaccia assume una larghezza minima tipica dei monitor da postazione di lavoro.	L'applicazione risulta difficilmente navigabile da dispositivi mobili (sovrapposizioni, tabelle tagliate). Rischio accettato in base ai casi d'uso tipici definiti nell'AdR.
Autenticazione Volatile (ADR-FE-6)	Conservazione del token in memoria (variabile di stato) per proteggerlo da vettori di attacco XSS comuni sul <code>localStorage</code> .	Una ricarica accidentale della scheda del browser chiude la sessione attiva, forzando l'utente a ripetere il login (peggioramento della <i>Quality of Life</i>).
Assenza di Retry Automatici Client-Side	Se una chiamata REST (es. configurazione del contesto, avvio task) fallisce per cause di rete, il frontend mostra un errore e richiede all'utente un'azione manuale (es. click su "Riprova"). L'unica eccezione automatizzata è la riconnessione del WebSocket.	Potenziata frustrazione dell'utente in caso di micro-disconnessioni, poiché deve reinviare attivamente il modulo. Evita tuttavia la complessità logica di gestire <i>idempotency keys</i> lato frontend in questa fase.
Validazioni Sincrone non esaustive	Le validazioni sui moduli (es. robustezza password, sintassi URL Git) forniscono un <i>feedback</i> rapido ma non duplicano la logica di business o i controlli di sicurezza completi del backend.	Un utente malintenzionato può bypassare la UI e inviare payload non validi all'API. Accettato perché il frontend non è considerato, per definizione, un confine di sicurezza (nessuna protezione WAF attiva).
Interattività Agente Changelog	L'MVP implementa la richiesta di input a metà esecuzione tramite eventi WS (<code>task.inputRequired</code>) e <code>POST /tasks/:id/input</code> . Essendo un rischio architetturale alto non validato dal PoC, in caso di esaurimento budget si adotterà il fallback al comportamento del PoC (decisione autonoma dell'agente e stub), sacrificando temporaneamente i requisiti RF.98–RF.102.	Parziale scostamento dall'AdR se il fallback si rende necessario, con l'agente che processerà tutti i task indiscriminatamente senza chiedere l'intervento umano per l'inserimento dello Sprint ID o l'esclusione.

Tabella 10: Matrice dei Compromessi e Rischi Accettati per il Frontend MVP.

13 Vincoli Ereditati dall'Infrastruttura AWS

Le sezioni della Specifica Operativa (Parte I) assumono come valide alcune proprietà fondamentali del sistema di esecuzione (ad esempio l'assenza di policy CORS o l'uso di URL relativi). Questa sezione rende esplicito il legame tra tali assunzioni e le decisioni prese nel documento di Progettazione Infrastruttura AWS. Raccogliere questi vincoli in un unico punto garantisce che un eventuale cambiamento futuro dell'infrastruttura comporti una revisione mirata di questa sola sezione, evitando disallineamenti silenti nel resto della progettazione.

13.1 Origine unica e assenza di policy CORS

L'infrastruttura AWS pone il frontend (ospitato su S3) e il backend (gestito tramite Application Load Balancer) dietro un'unica distribuzione CloudFront condivisa. Il comportamento di routing instrada le richieste ai percorsi API (`/api/*` e `/socket.io/*`) verso il backend, e tutto il resto verso l'artefatto statico del frontend. Poiché frontend e backend condividono lo stesso dominio agli occhi del browser, nessuna richiesta effettuata dall'interfaccia è considerata *cross-origin*. Di conseguenza, il frontend non richiede alcuna configurazione, gestione o negoziazione di policy CORS, eliminando alla radice un'intera classe di potenziali anomalie di rete.

13.2 Routing lato client e Custom Error Response

L'applicazione frontend gestisce il routing interamente lato client (tramite **TanStack Router**). Una richiesta diretta del browser a un percorso specifico (es. `/reports/abc123`) raggiungerebbe S3, che non possedendo tale file restituirebbe un errore 403/404. L'infrastruttura risolve questo comportamento configurando su CloudFront una *Custom Error Response* che intercetta questi errori e restituisce il file `index.html` con stato HTTP 200. Per lo sviluppo frontend, questo vincolo impone che ogni rotta protetta o pubblica sia in grado di auto-risolversi e di leggere lo stato di autenticazione in modo sincrono direttamente al caricamento (come garantito dalle guardie di rotta lette dal `sessionStore`), permettendo il corretto funzionamento dell'accesso diretto o della ricarica della pagina tramite browser.

13.3 Configurazione a build-time per risorse statiche

A differenza del backend, che riceve la configurazione a *runtime* tramite variabili d'ambiente iniettate nel container, il frontend è distribuito come artefatto statico puro. Variabili come l'indirizzo base delle API (`VITE_API_BASE_URL`) vengono risolte in fase di compilazione (*build-time*) e integrate irreversibilmente nel pacchetto JavaScript. Per supportare deploy multi-ambiente senza dover ricompilare artefatti diversi, l'infrastruttura impone l'uso di percorsi relativi (es. `/api/v1`). Il frontend non deve quindi mai concatenare domini assoluti nelle chiamate ai servizi interni.

13.4 Requisiti per connessioni WebSocket

Il canale realtime bidirezionale necessario per monitorare l'avanzamento delle Task è supportato nativamente dall'infrastruttura tramite specifiche policy CloudFront (che inoltrano gli header `Upgrade` e `Connection`) e configurazioni dell'Application Load Balancer (che implementa *stickiness* e *idle timeout* estesi). Questo solleva il frontend dal dover implementare logiche complesse di *failover* o *long-polling* di fallback: una semplice logica di riconnessione automatica del *socket* è sufficiente a garantire la resilienza, in quanto l'infrastruttura stessa mantiene la connessione aperta durante l'esecuzione di agenti a lunga durata.

13.5 Streaming dei file ed Esportazione PDF

Il bucket S3 designato per ospitare gli artefatti applicativi (come i report PDF generati) è strettamente privato e accessibile unicamente dai container del backend tramite VPC Endpoint. Questo vincolo di sicurezza impedisce categoricamente al frontend di tentare l'accesso diretto ai file tramite URL pubbliche o di generare *presigned URL*. Il comando di esportazione del PDF nel frontend deve consumare un endpoint REST del backend che scarica l'oggetto da S3 e lo trasmette in *streaming* nel corpo della risposta HTTP same-origin. L'indicatore di caricamento sulla UI deve coprire l'intera durata di questo transito.

13.6 Gestione degli errori di rete e Sicurezza Trasversale

L'infrastruttura AWS accetta determinati rischi a fronte di un forte contenimento dei costi e dei tempi di rilascio dell'MVP. Questi si riflettono direttamente sulla gestione errori e sulla sicurezza del frontend:

- **SPOF del NAT Gateway:** L'uso di un singolo NAT Gateway per far dialogare il backend con GitHub introduce un punto singolo di fallimento di rete. Se questa connettività si interrompe, l'errore che l'utente visualizza nel frontend ricade nella famiglia degli errori generici di sistema, e non deve innescare falsi avvisi di "Credenziale GitHub non valida".
- **Trasporto in chiaro interno:** Il perimetro MVP esclude l'adozione di un dominio custom, costringendo l'Application Load Balancer a operare senza certificato TLS pubblico (ricevendo traffico da CloudFront in chiaro). Questo vincolo infrastrutturale giustifica pienamente la decisione architetturale del frontend (ADR-FE-6) di mantenere il *JSON Web Token* esclusivamente in memoria, evitando ulteriori esposizioni sul client.
- **Assenza di Web Application Firewall (WAF):** In mancanza di un WAF perimetrale, le validazioni condotte dal frontend (formati, lunghezze, correttezza sintattica) sono classificate puramente come ausili per la *User Experience* (UX). Non costituiscono una barriera di sicurezza, e non si deve assumere che il backend sia esentato dal ri-validare strettamente ogni singolo payload ricevuto.

Vincolo per il frontend	Origine Architetturale	Impatto sullo Sviluppo
Nessuna gestione delle policy CORS	Distribuzione CloudFront condivisa per routing traffico statico (S3) e dinamico (ALB).	Tutte le chiamate API e WebSocket risultano same-origin. Nessun preflight o gestione header CORS necessaria lato client.
Risoluzione rotta garantita ad ogni punto di ingresso	Custom Error Response configurata su CloudFront (403/404 da S3 reindirizzati a <code>index.html</code>).	Le guardie di rotta devono poter validare lo stato (<code>isAuthenticated</code> , <code>credentialsStatus</code>) immediatamente al caricamento diretto di un URL profondo.
Path API relativo	Artefatti su S3 immutabili a <i>runtime</i> ; <code>VITE_API_BASE_URL</code> compilata a <i>build-time</i> .	Il client usa esclusivamente prefissi relativi (es. <code>/api/v1</code>) per non legarsi a un dominio statico, agevolando le pipeline CI/CD.
Autenticazione WebSocket all'handshake	CloudFront <i>Origin Request Policy (Forward All)</i> e ALB con <i>stickiness</i> abilitata.	La libreria Socket.IO può autenticarsi al momento dell'apertura inviando header custom senza dipendere da riconessioni complesse o long-polling.
Download dei PDF interamente mediato	Bucket S3 degli artefatti privato e inaccessibile da Internet pubblico (solo via VPC Endpoint).	I PDF non vengono mai referenziati o aperti con URL esterne. Il frontend attende il completamento dello streaming binario dal backend.
Validazioni client-side a puro scopo UX	Assenza di Web Application Firewall nel perimetro MVP.	Il frontend evita chiamate di rete inutili bloccando input errati (es. email malformate), ma la validazione formale e la sicurezza restano in capo al backend.

Tabella 11: Riepilogo dei vincoli tecnici imposti al frontend dalle decisioni infrastrutturali.